

FPGA Overlay Driver

不依賴 PYNQ 的 Bitstream 載入工具

設計討論文件 · 2025

1. 背景與動機

PYNQ 是 Xilinx 為 Zynq 系列設計的 Python 框架，提供了方便的 overlay 機制讓使用者從 Python 載入 bitstream 並操作 PL 上的 IP Core。然而 PYNQ 的框架層次偏厚，相依性重，在以下情境下會造成不便：

- 需要在沒有 PYNQ 環境的 Linux 系統上部署
- 希望避免 Python overhead，以 C 或其他語言直接操控硬體
- 需要更精細地控制 bitstream 載入流程
- 在 BCI 等即時性要求較高的應用場景中，Python 框架的延遲不可接受

本工具的目標是：以 C Userspace Library 的形式，重新實作 PYNQ overlay 的核心功能，讓 .bit 與 .hwh 檔案可以直接透過 Linux OS 介面載入，完全不依賴 PYNQ。

2. 專案目標

核心功能如下：

- 將 Vivado 產出的 .bit 檔轉換為 Linux FPGA Manager 可接受的 .bin 格式
- 解析 .hwh 檔取得各 IP Core 的記憶體映射位址
- 透過 Linux sysfs 介面（/sys/class/fpga_manager）載入 bitstream
- 以 /dev/mem + mmap 直接讀寫 PL 暫存器
- 提供乾淨的 CLI 介面供使用者操作

非目標（此階段不實作）：

- Kernel Module (Layer 3)
- Device Tree Overlay 自動化
- DMA 或中斷處理

3. 開發環境

3.1 硬體

項目	規格
開發驗證板	PYNQ-Z2
晶片	Zynq-7000 XC7Z020
ARM Core	Cortex-A9 (32-bit)
目標部署板	Kria KV260
KV260 晶片	Zynq Ultrascale+ XCK26
KV260 ARM Core	Cortex-A53 (64-bit)

3.2 軟體環境

項目	說明
開發方式	Windows 上 VS Code 撰寫，SSH 進 PYNQ-Z2 編譯執行
編譯器	板子上的 gcc (ARM native)，不需 cross-compile
OS	PYNQ-Z2 上的 Ubuntu Linux
Vivado 工具	產生 .bit 與 .hwh，在 Windows 主機上運行
JTAG	不使用 (JTAG 用於 Vivado 燒錄，本工具不涉及)

3.3 開發流程

日常開發的工作流程如下：

```
# 1. 在 Windows 上寫好 .c 檔
# 2. SCP 傳到板子
scp overlay.c xilinx@192.168.2.99:~/

# 3. SSH 進板子編譯
ssh xilinx@192.168.2.99
gcc overlay.c -o overlay

# 4. 執行測試
./overlay load design.bit design.hwh
```

4. 技術背景

4.1 PYNQ 的現有做法

PYNQ overlay 載入流程（本工具要繞過的部分）：

- 解析 .hwh (XML) → 取得 IP address map
- .bit → .bin (bitstream byte-swap)
- 寫入 /lib/firmware/
- 呼叫 Linux FPGA Manager sysfs 介面
- 產生 Python driver 物件供使用者操作

本工具以 C 實作以上所有步驟，移除 Python 相依性。

4.2 Linux FPGA Manager

Linux kernel 內建 fpga_manager framework，提供 sysfs 介面：

```
/sys/class/fpga_manager/fpga0/flags      # 0=full, 1=partial
/sys/class/fpga_manager/fpga0/firmware    # 寫入 .bin 檔名觸發載入
/sys/class/fpga_manager/fpga0/state      # 確認載入狀態
```

bitstream 檔案須先放至 /lib/firmware/ 目錄，FPGA Manager 才能讀取。

4.3 .hwh 檔案格式

.hwh 是 Vivado 輸出的 XML 檔，記錄 IP Core 的完整資訊：

```
<MODULE FULLNAME="design_1/axi_gpio_0"
    VLNV="xilinx.com:ip:axi_gpio:2.0">
  <REGISTERS>
    <PROPERTY NAME="BASEADDR" VALUE="0x41200000"/>
    <PROPERTY NAME="HIGHADDR" VALUE="0x4120FFFF"/>
  </REGISTERS>
</MODULE>
```

5. 實作架構

5.1 模組拆分

模組	檔案	功能
Bitstream 轉換	bit2bin.c	剝除 .bit header，byte-swap 產生 .bin
HWH 解析	hwh_parser.c	解析 XML，取得各 IP 的 base/high address

模組	檔案	功能
FPGA 載入	fpga_load.c	操作 sysfs，觸發 FPGA Manager 載入
MMIO 存取	mmio.c	open /dev/mem + mmap，讀寫 PL 暫存器
CLI 入口	main.c	整合以上模組，提供命令列介面

5.2 CLI 介面設計

```

./overlay load  design.bit design.hwh  # 載入 bitstream
./overlay list                                # 列出所有 IP Core 與位址
./overlay write axi_gpio_0 0x00 0xFF    # 寫暫存器
./overlay read  axi_gpio_0 0x00         # 讀暫存器

```

5.3 預期輸出範例

```

$ ./overlay load design.bit design.hwh
[✓] Converted design.bit → design.bin
[✓] Parsed 3 IP cores from design.hwh
[✓] Bitstream loaded, FPGA state: operating

$ ./overlay list
IP Cores:
  axi_gpio_0      @ 0x41200000
  axi_timer_0     @ 0x42800000
  my_dsp_core     @ 0x43C00000

```

6. 開發層次選擇

本專案採用 Layer 2（C Userspace Library）實作，以下為三個層次的比較：

	Layer 1 Shell Script	Layer 2 C Userspace (本專案)	Layer 3 Kernel Module
難度	低	中	高
Debug	容易	容易 (gdb/printf)	困難 (crash 死整台機器)
即時性	差	普通	好
中斷處理	不支援	有限支援	完整支援
DMA	不支援	困難	原生支援

	Layer 1 Shell Script	Layer 2 C Userspace (本專案)	Layer 3 Kernel Module
適合情境	快速驗證	研究工具、lab 內部使用	產品化、系統級貢獻

選擇 Layer 2 的理由：此工具定位為 lab 內部使用的工程工具，不需要 kernel 級的即時性或 DMA 支援，開發速度優先。

7. 移植計畫：PYNQ-Z2 → KV260

7.1 主要差異

項目	PYNQ-Z2	KV260
ARM 架構	32-bit Cortex-A9	64-bit Cortex-A53
AXI Address 寬度	uint32_t	uint64_t
FPGA Manager sysfs	路徑相同	路徑相同
.hwh 格式	相同	相同
Bitstream	XC7Z020 專用	XCK26 專用，需重新合成
TrustZone	無	有，部分記憶體區域受限

7.2 移植改動

從一開始就準備好移植，改動量可以很小：

```
// config.h
#ifdef ZYNQ7000
    typedef uint32_t fpga_addr_t;
#else // Ultrascale+
    typedef uint64_t fpga_addr_t;
#endif
```

主要改動集中在一個 config.h，其餘邏輯不需修改。

8. 建議實作順序

- Step 1：手動用 shell 命令確認 FPGA Manager 環境正常
- Step 2：實作 bit2bin.c，確認 bitstream 轉換正確
- Step 3：實作 fpga_load.c，讓 C 程式能成功載入 bitstream
- Step 4：實作 hwh_parser.c，能列出 IP Core 位址

- Step 5：實作 mmio.c，能讀寫 PL 暫存器
- Step 6：整合成 CLI，加上錯誤處理
- Step 7：在 PYNQ-Z2 完整驗證後，移植至 KV260

每個步驟獨立可驗證，降低一次踩多個坑的風險。

文件版本 v1.0 · 基於設計討論整理